



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : M Raihan Al Lutfi

NPM : 2428240156

Kelas : SI5C

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	M Raihan AL Lutfi
NPM	2428240156

Kelas	SI5C
Pertemuan Ke-	3 - 4
Topik / Judul Praktikum	Membangun RESTful API dengan Express.js
Tanggal Pelaksanaan	21 – 22 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Mahasiswa mampu membuat backend RESTful API menggunakan Express.js.

II. Dasar Teori

REST (Representational State Transfer) merupakan gaya arsitektur untuk membangun layanan berbasis jaringan dengan memanfaatkan protokol HTTP. Data pada RESTful API direpresentasikan sebagai resource dan pertukaran data pada praktikum ini menggunakan format JSON.

Node.js digunakan sebagai runtime untuk menjalankan JavaScript di sisi server, sedangkan Express.js digunakan sebagai framework untuk membuat server dan routing API. Express menyediakan routing GET, POST, PUT, dan DELETE serta middleware `express.json()` untuk membaca request body JSON.

Pada studi kasus Area Parkir, resource yang digunakan adalah Catatan Parkir dengan endpoint `/parking-records`. Data memiliki atribut `id`, `platNomor`, `jenisKendaraan`, `waktuMasuk`, `waktuKeluar`, dan `biaya`.

GET digunakan untuk membaca data, POST untuk menambahkan data, PUT untuk mengganti data berdasarkan ID, dan DELETE untuk menghapus data berdasarkan ID. Filtering dilakukan menggunakan query string, misalnya `/parking-records?jenisKendaraan=motor`.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v24.16.0
2	Visual Studio Code	1.138.0

3	Github Desktop	3.6.6 (x64)
4	Postman	v12.30.0

IV. Langkah Kerja

1. Membuat folder project RESTful API Area Parkir, kemudian melakukan inisialisasi Node.js menggunakan `npm init -y` dan membuka project melalui Visual Studio Code.

2. Menginstal Express.js sebagai framework untuk membuat RESTful API dan Nodemon sebagai development dependency agar server dapat melakukan restart otomatis ketika kode mengalami perubahan.

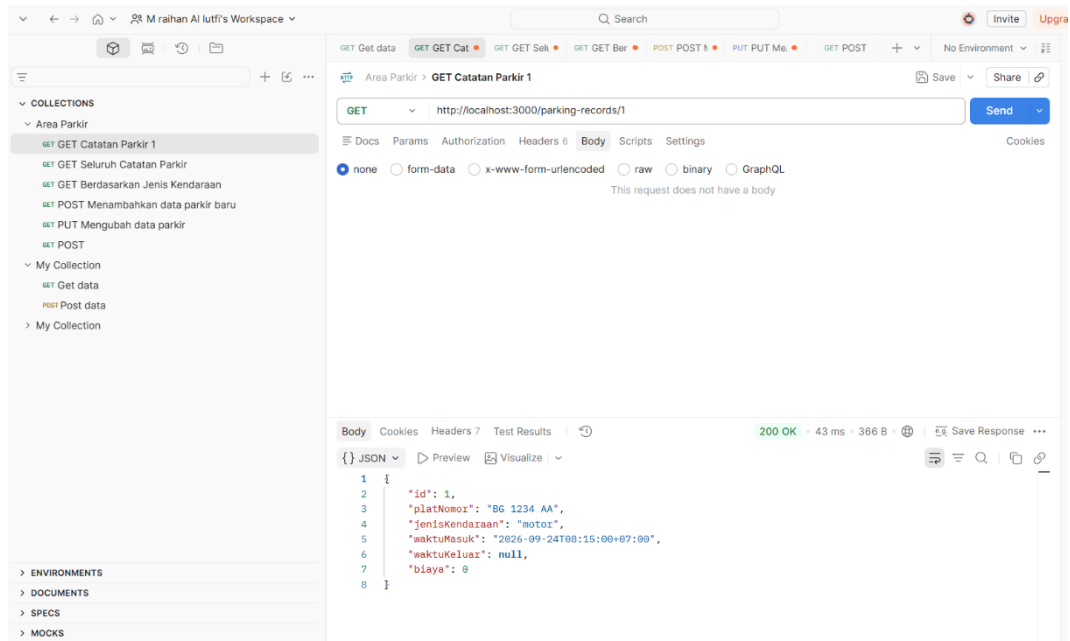
3. Membuat file `app.js`, mengimpor Express.js, mengatur port server, serta menambahkan `express.json()` agar server dapat menerima request dalam format JSON. Selanjutnya membuat array `parkingRecords` yang berisi data awal catatan parkir.

4. Membuat endpoint GET, POST, PUT, dan DELETE pada resource `/parking-records`. Endpoint GET juga dilengkapi dengan pencarian berdasarkan ID dan filtering berdasarkan jenisKendaraan.

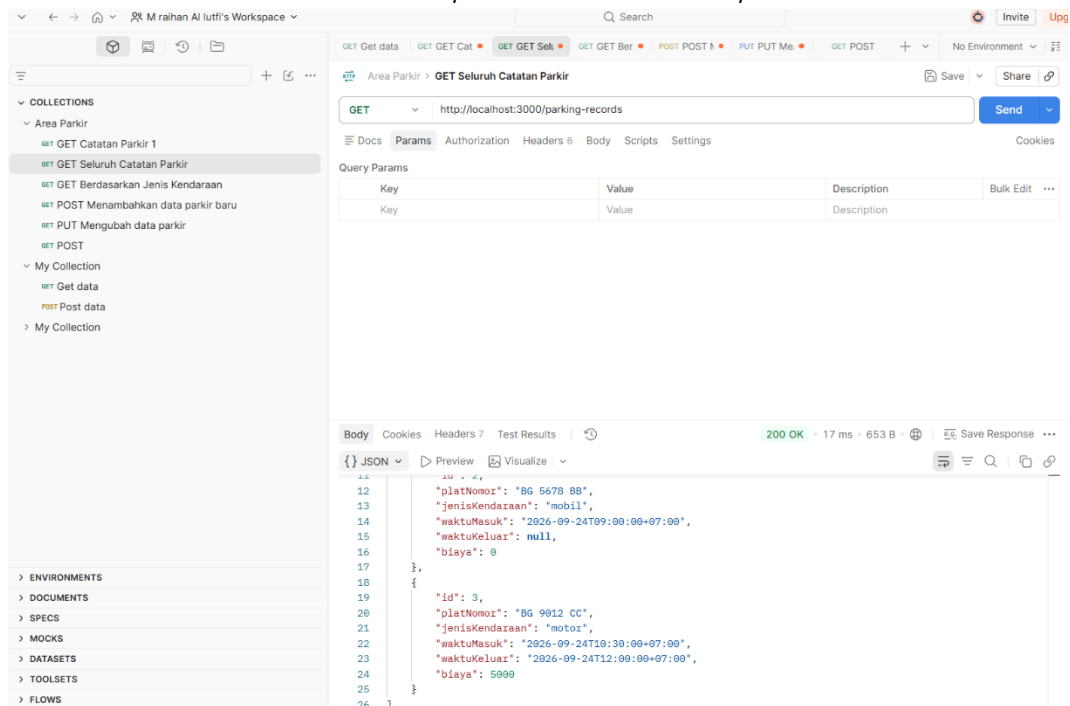
5. Menambahkan validasi pada data yang dikirim melalui POST dan PUT, serta membuat response 404 apabila data atau endpoint tidak ditemukan. Response API dibuat dalam format JSON.

6. Menjalankan server menggunakan `npm run dev`, kemudian melakukan pengujian endpoint menggunakan Postman atau Thunder Client, meliputi pengambilan data, filtering, penambahan, perubahan, dan penghapusan data catatan parkir.

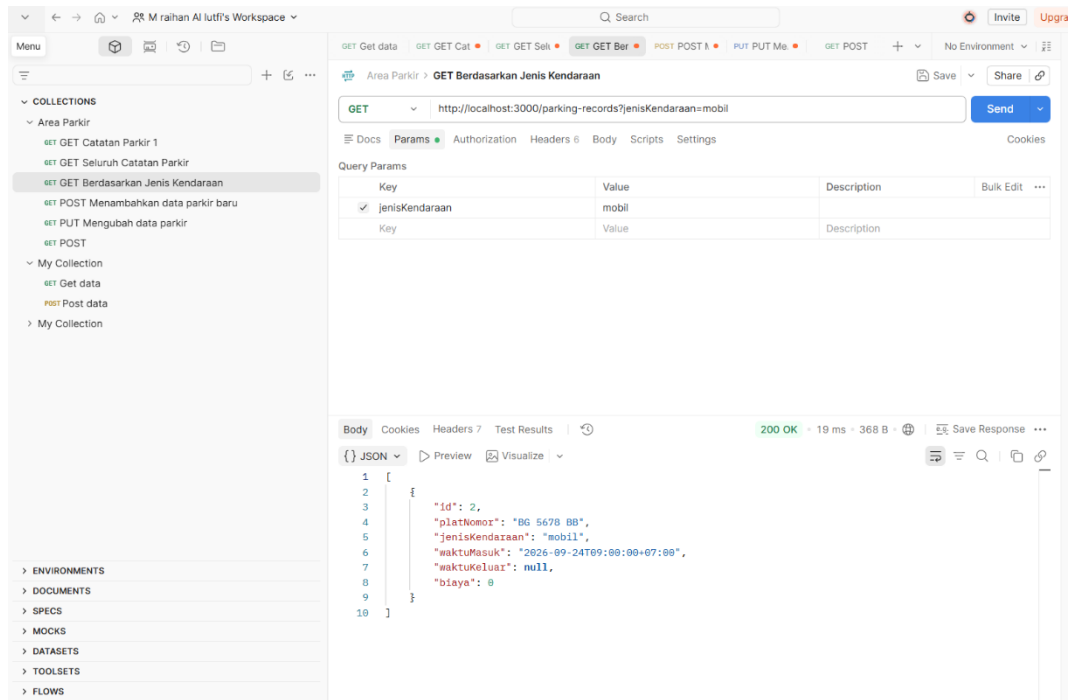
V. Hasil dan Pembahasan



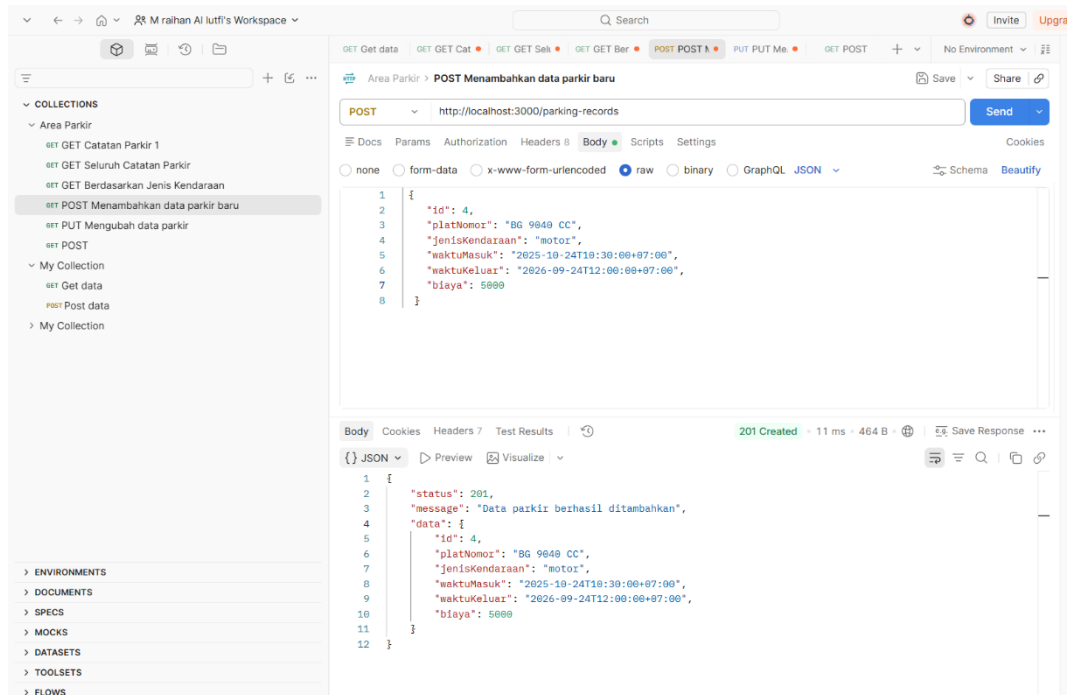
Gambar 1. untuk Menampilkan Seluruh Data Parkir pada Postman



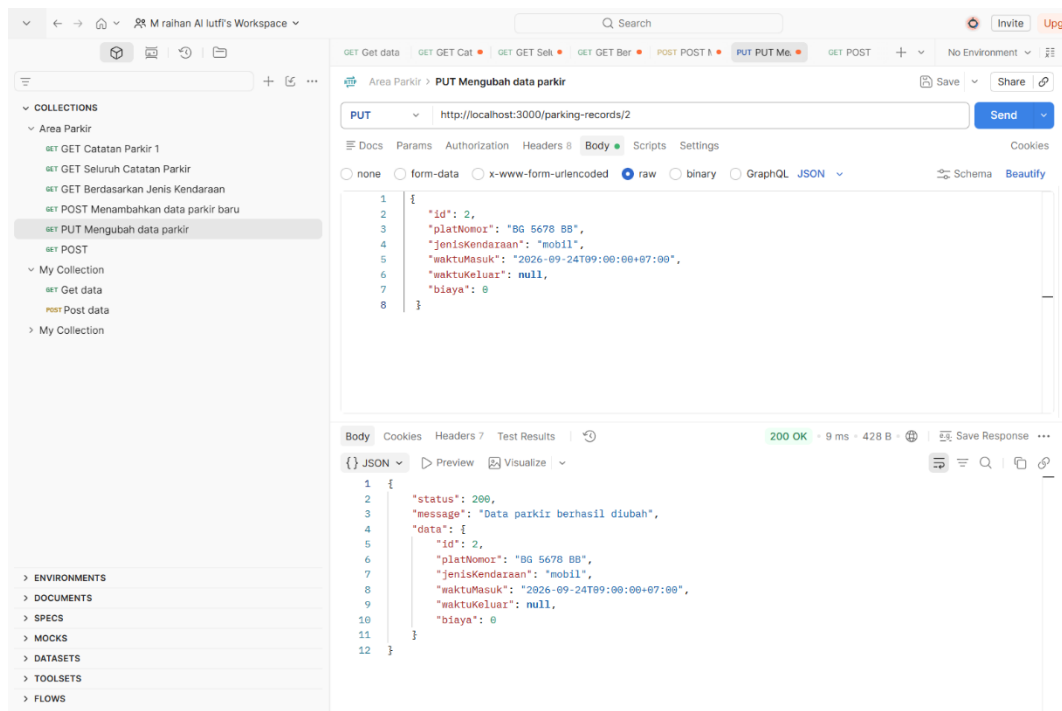
Gambar 2. untuk Menampilkan Seluruh Data Parkir pada Postman



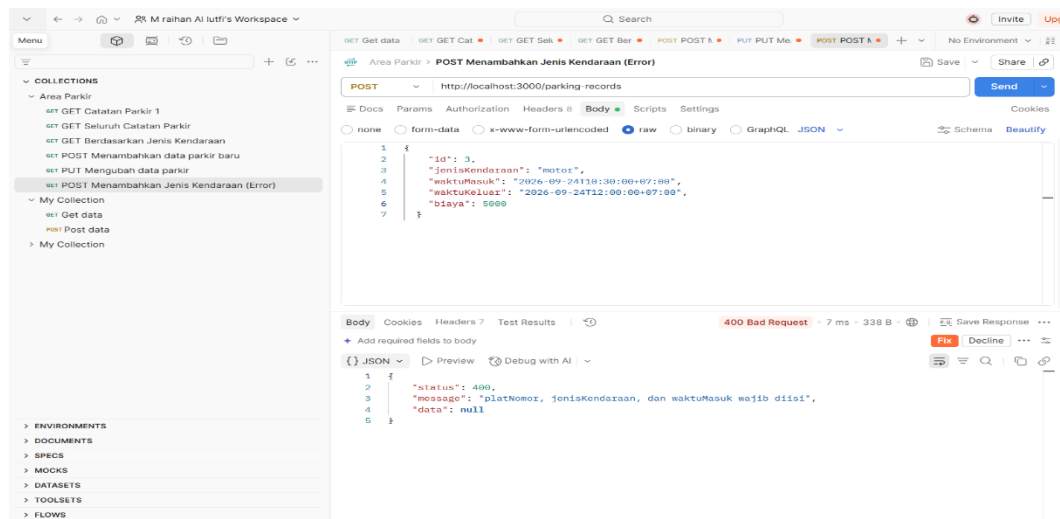
Gambar 3. untuk Menampilkan Data Berdasarkan Jenis Kendaraan pada Postman



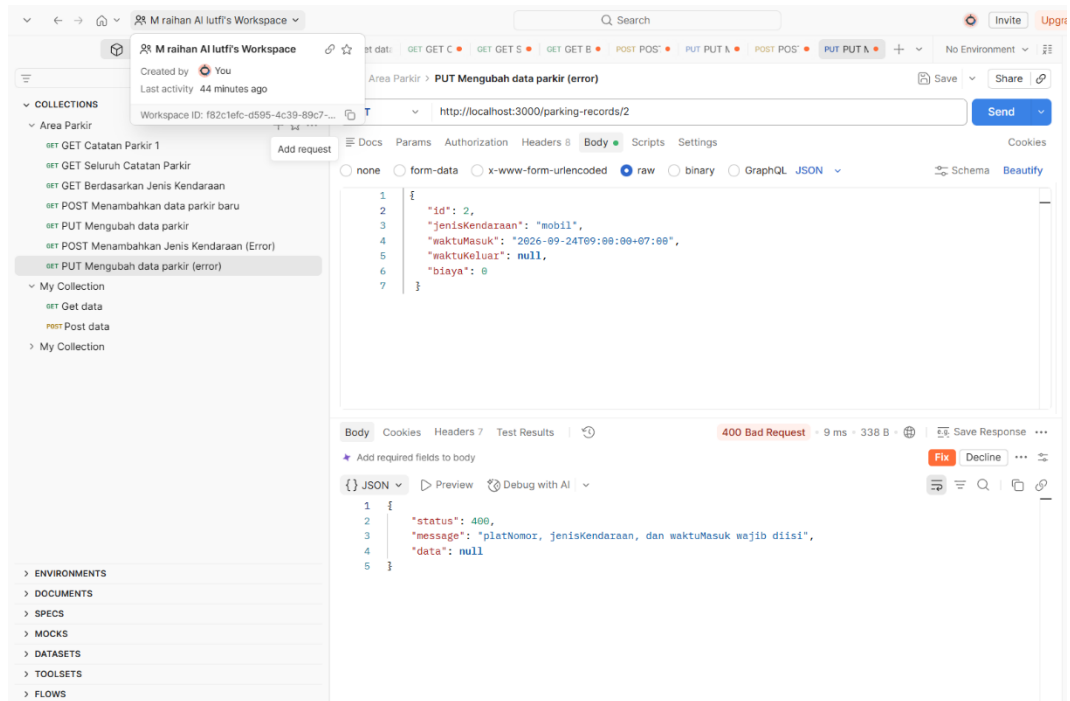
Gambar 4. untuk Menambahkan Data Parkir Baru di Postman



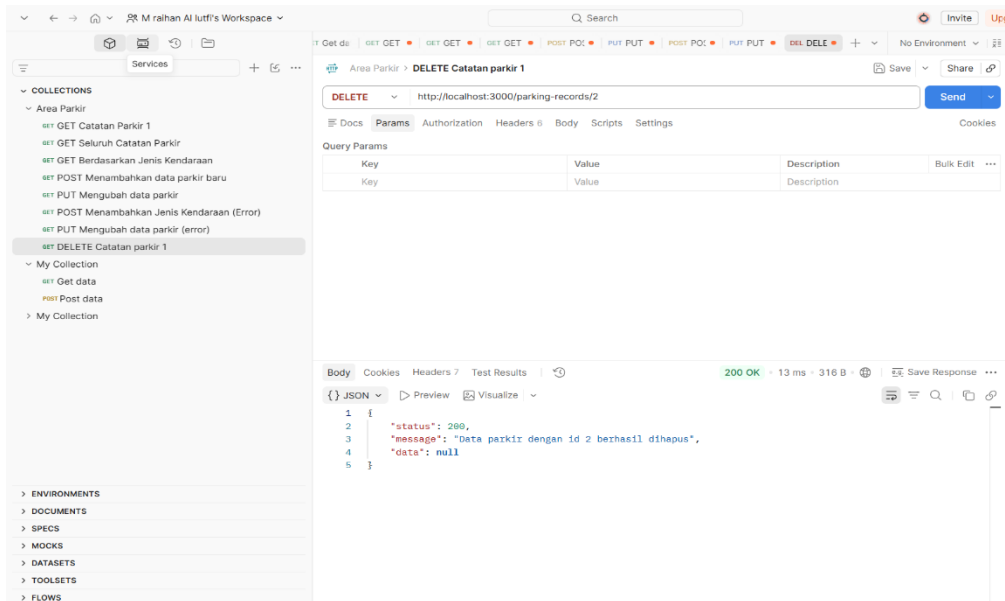
Gambar 5. untuk Mengubah Data Parkir di Postman



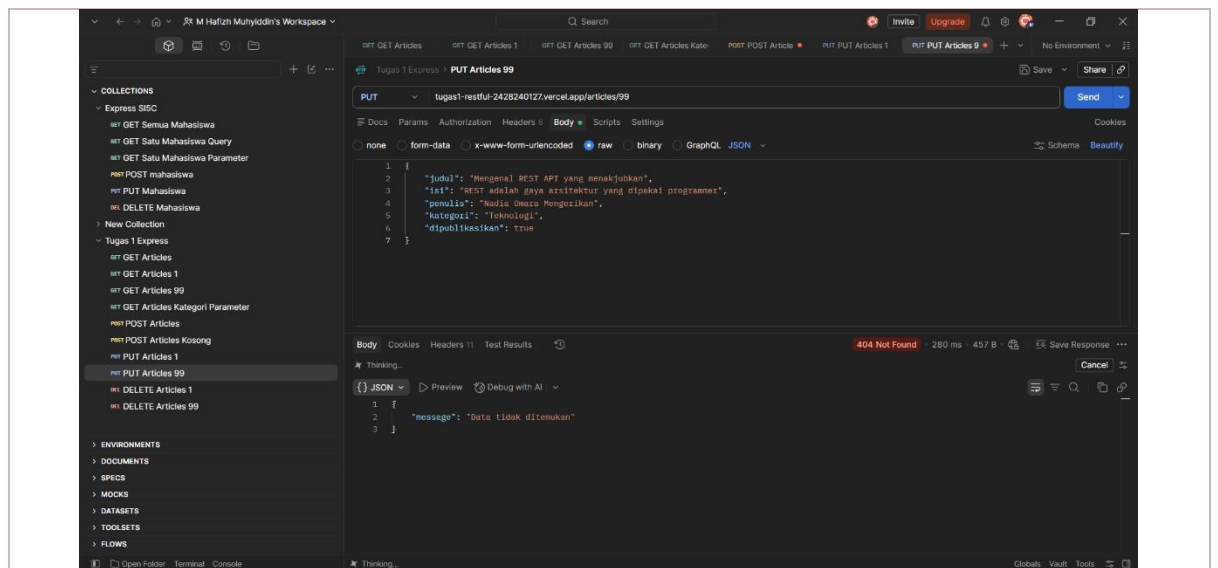
Gambar 6. untuk Menambahkan Jenis Kendaraan Yang Error di Postman



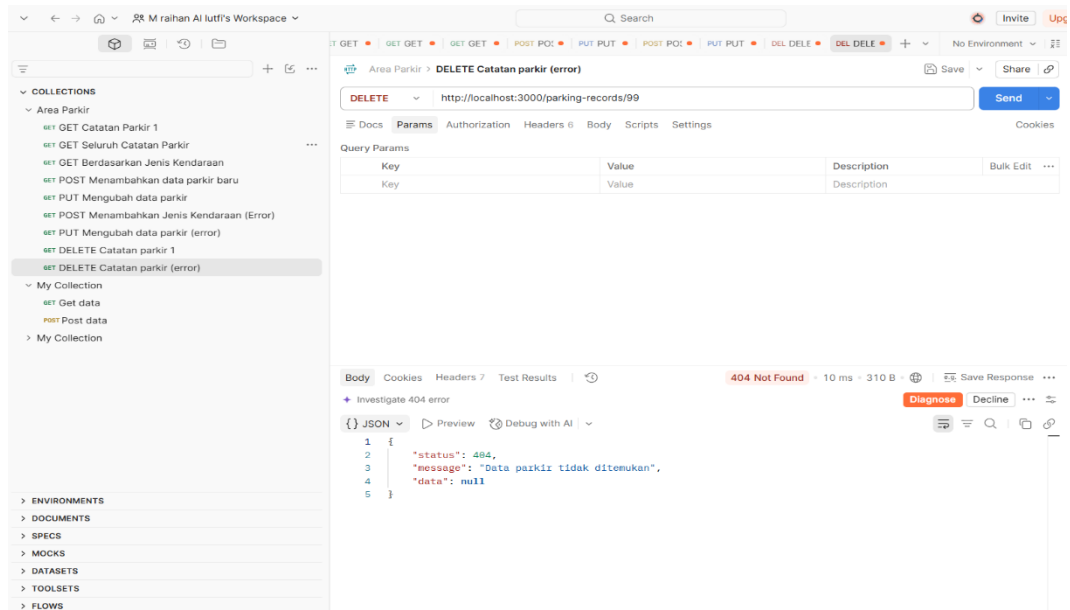
Gambar 7. Mengubah data parkir yang tidak ada di postman (error)



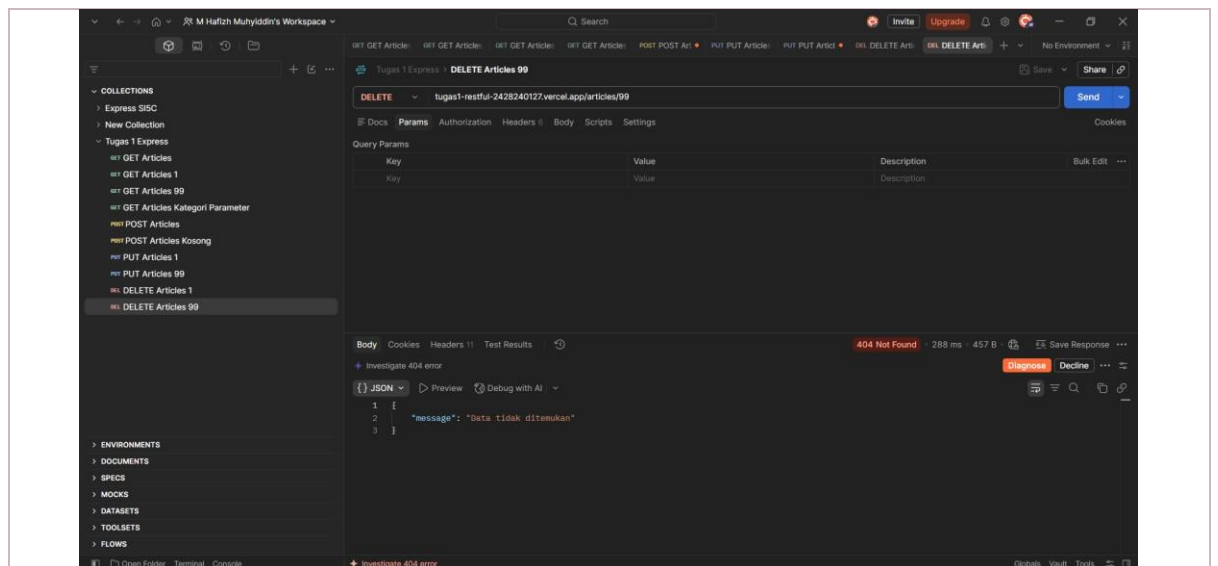
Gambar 8. untuk Mendelete Data Parkir di Postman



Gambar 8. Pengujian Endpoint PUT /articles/99 untuk mengubah Data Artikel yang tidak ada di Postman



Gambar 9. Pengujian Endpoint DELETE /articles/1 untuk menghapus Data Artikel di Postman



Gambar 10. Pengujian Endpoint DELETE /articles/99 untuk menghapus Data Artikel yang tidak ada di Postman

Pembahasan:

Berdasarkan pengujian yang dilakukan menggunakan Postman pada RESTful API Express.js untuk pengolahan data artikel (Topik 16: Blog) yang berjalan pada lingkungan lokal maupun *serverless* Vercel, seluruh skenario telah berjalan sesuai aturan RESTful API. Pengujian diawali dengan pemanggilan metode GET untuk mengambil seluruh data artikel serta data spesifik berbasis parameter ID. Metode GET dipilih karena bersifat *read-only* (*safe* dan *idempotent*), di mana server merespons dengan kode status 200 OK dan menyertakan data berformat JSON.

Pada skenario pencarian data yang tidak terdaftar, pemanggilan GET dengan ID yang tidak eksis diuji untuk memastikan keandalan *error handling*. Server secara tepat mengembalikan kode status 404 Not Found beserta *response body* pesan kesalahan berformat JSON. Selain itu, fitur penyaringan (*filtering*) data berbasis *query string* pada *endpoint* GET berhasil memfilter data secara dinamis berdasarkan parameter kategori dan merespons dengan kode status 200 OK.

Pengujian dilanjutkan pada penambahan data artikel baru menggunakan metode POST. Penggunaan POST bertujuan untuk membentuk *resource* baru pada server, di mana request yang menyertakan *payload* JSON lengkap berhasil diproses, diberi ID otomatis, dan menghasilkan respons 201 Created. Untuk menguji validasi *input*, request POST dikirimkan tanpa menyertakan *field* wajib, dan server berhasil mencegah permintaan tersebut dengan mengembalikan kode status 400 Bad Request.

Pada operasi pembaruan data, metode PUT digunakan untuk memperbarui seluruh atribut artikel secara utuh. Request PUT pada ID yang valid menghasilkan status 200 OK beserta data terbaru, sedangkan pada ID yang tidak terdaftar secara konsisten mengembalikan status 404 Not Found. Hal serupa berlaku pada metode DELETE untuk menghapus *resource* artikel, di mana pengujian pada ID valid berhasil menghapus data dan mengembalikan status respons sukses, sementara pengujian pada ID non-eksis menghasilkan respons 404 Not Found.

Selama proses pengembangan dan pengujian, kendala yang dihadapi terletak pada pembacaan *request body* JSON serta penyesuaian *routing serverless* saat di-deploy ke Vercel yang sempat menyebabkan galat *routing*. Solusi yang diterapkan adalah menambahkan *middleware* `express.json()` pada kode Express.js serta menyusun berkas konfigurasi `vercel.json` secara tepat agar seluruh alur *endpoint* API dapat terarah dan dieksekusi dengan sempurna di platform Vercel.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	-	-

VII. Kesimpulan

Secara keseluruhan, pembuatan *backend* RESTful API murni berbasis Express.js untuk data artikel telah berhasil diimplementasikan tanpa menggunakan *template engine*. Seluruh *endpoint* CRUD beserta fitur *filtering query string* telah memenuhi konvensi RESTful API dengan penerapan HTTP status code seperti 200 OK, 201 Created, 400 Bad Request, dan 404 Not Found secara konsisten, baik di lingkungan lokal maupun setelah di-deploy ke platform Vercel.

Daftar Pustaka

Crockford, D. (2006). *RFC 4627: The application/json Media Type for JavaScript Object Notation (JSON)*. IETF.

- Fielding, R. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.
- Herron, D. (2020). *Node.js Web Development: Master the usability of Node.js to build scalable web applications* (5th ed.). Packt Publishing.
- Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
- Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. *IEEE Internet Computing*, 14(6), 80-83.
-

Lampiran

Tautan Repository GitHub:

<https://github.com/hadinnn/tugas1-restful-2428240127>

Tautan Deployment (jika ada):

tugas1-restful-2428240127.vercel.app/articles
